

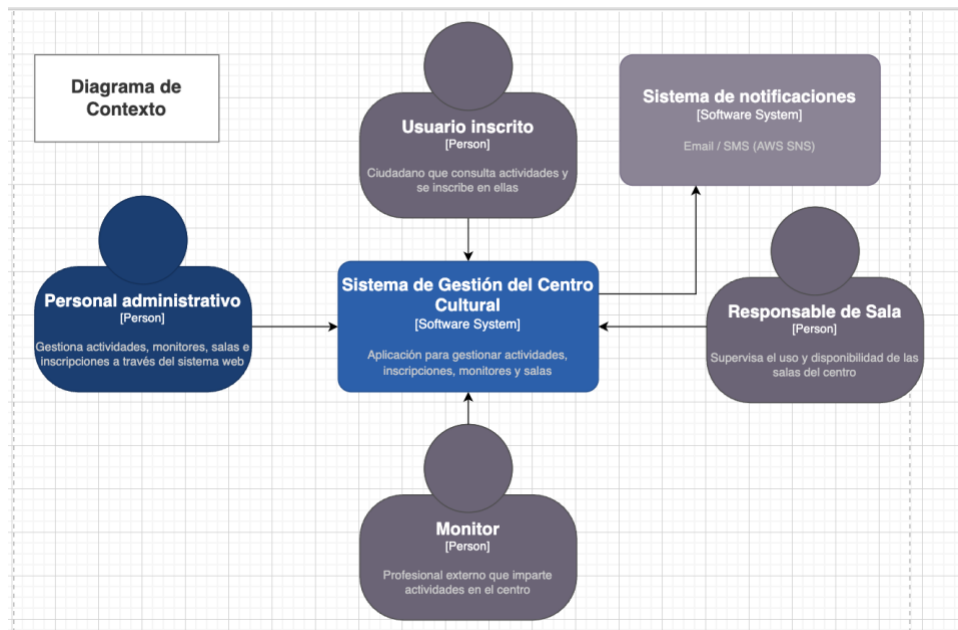
Práctica Final Arquitectura del Software



Universidad
Pontificia
de Salamanca

1. Arquitectura para Plataforma de Gestión de Actividades para un Centro Cultural:

1.1 Diagrama de Contexto:



Sistema Central:

- **Sistema de Gestión del Centro Cultural [Software System]**: Es el núcleo de la solución de software propuesta. Actúa como la plataforma centralizada que permite digitalizar y orquestar toda la operativa diaria del centro, desde la publicación de la oferta de actividades hasta la reserva de espacios físicos y el control de aforos.

Actores (Usuarios del Sistema):

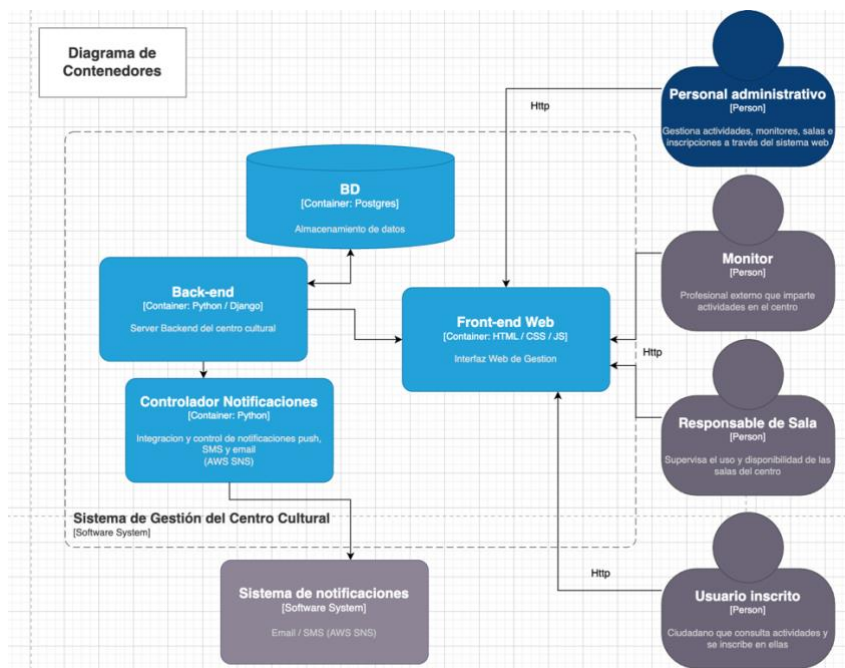
- **Personal Administrativo [Person]**: Perfil con privilegios de gestión (*Back-Office*). Es el actor principal encargado de alimentar el sistema: crea el catálogo de actividades, da de alta a los monitores, configura la disponibilidad de las salas y supervisa las inscripciones generales para auditar y evitar solapamientos o duplicidades.
- **Usuario Inscrito [Person]**: Representa al ciudadano o cliente final (*Front-Office*). Interactúa con el sistema para consultar la cartelera de talleres y eventos disponibles, visualizar horarios y gestionar sus propias inscripciones de forma totalmente autónoma.
- **Monitor [Person]**: Profesional (interno o externo) contratado para impartir las actividades. Accede a la plataforma exclusivamente para consultar sus horarios asignados, revisar los grupos de usuarios a su cargo y verificar las salas donde debe presentarse.

- **Responsable de Sala [Person]:** Perfil operativo centrado en la logística física del edificio. Utiliza el sistema en modo consulta para verificar la ocupación en tiempo real de los espacios, asegurando que las salas estén disponibles y acondicionadas según las reservas registradas.

Sistemas Externos:

- **Sistema de Notificaciones [Software System]:** Servicio de infraestructura de terceros (apoyado en tecnologías como AWS SNS) que se integra en la arquitectura para externalizar el envío asíncrono de comunicaciones transaccionales. Es el responsable de despachar correos electrónicos o SMS (ej. confirmaciones de inscripción o avisos de cancelación) garantizando la entrega sin bloquear los procesos del sistema central.

1.2 Diagrama de contenedores:



El Nivel 2 de la arquitectura C4 (Diagrama de Contenedores) ilustra cómo el sistema monolítico se ha desacoplado en unidades de despliegue independientes, separando la interfaz de usuario, la lógica de negocio, la persistencia y las tareas asíncronas para maximizar el rendimiento y la escalabilidad

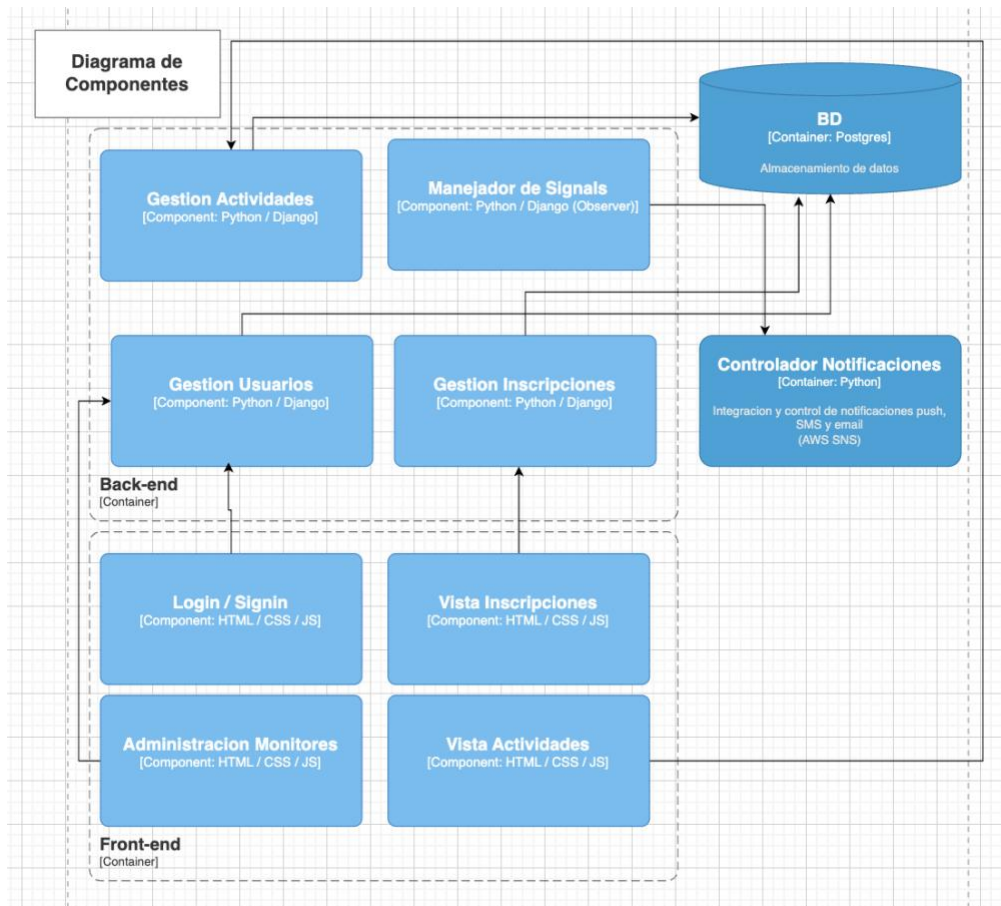
Contenedores del Sistema (El Núcleo):

- Front-end Web [Container: HTML / CSS / JS]: Actúa como la capa de presentación y punto de acceso unificado para todos los actores del sistema (Personal, Monitores, Responsables y Usuarios). Es responsable de renderizar la interfaz gráfica de usuario (GUI), capturar los eventos y comunicarse con el servidor a través de peticiones HTTP, ofreciendo una experiencia interactiva fluida.
- Back-end [Container: Python / Django]: Es el motor lógico de la arquitectura. Recibe las peticiones del Front-end, procesa la lógica de negocio central (validación de disponibilidad de aforos, control de solapamiento de horarios en las salas, autenticación de usuarios) y actúa como puente de orquestación entre la base de datos y los servicios auxiliares.
- BD [Container: PostgreSQL]: Contenedor de persistencia de datos relacional. Se encarga de almacenar de forma segura e íntegra toda la información del dominio: el catálogo de actividades, los perfiles de usuario, el estado de las salas y el registro histórico de inscripciones. Garantiza la consistencia de los datos mediante restricciones de integridad referencial (*Foreign Keys*).
- Controlador Notificaciones [Container: Python]: Proceso independiente en segundo plano (tipo *Worker* asíncrono). Su responsabilidad exclusiva es recoger las órdenes de aviso emitidas por el Back-end y gestionar la comunicación con el proveedor externo. Aislar este contenedor permite que, si la red es lenta o la API externa falla, el sistema principal no se bloquee ni penalice el tiempo de respuesta al usuario.

2. Actores y Sistemas Externos (El Entorno):

- Actores (Personal administrativo, Monitor, Responsable de Sala, Usuario inscrito): Interactúan exclusivamente con el Front-end Web a través de la red (HTTPS), cada uno visualizando interfaces y menús adaptados a su rol y permisos.
- Sistema de notificaciones [Software System]: Servicio de terceros (AWS SNS) que recibe las cargas de trabajo (*payloads*) estructuradas directamente desde el Controlador de Notificaciones para despachar los emails y SMS finales a los usuarios.

1.3 Diagrama de Componentes:



Componentes del Contenedor: Back-end (Lógica de Negocio y Datos)

Estos componentes agrupan las responsabilidades de los Modelos (models.py) y las Vistas (views.py) del patrón MVT de Django. Son el cerebro del sistema:

- **Gestión Actividades** [Component: Python / Django]: Módulo central que encapsula la lógica CRUD (Crear, Leer, Actualizar, Borrar) del catálogo del centro cultural. Es responsable de ejecutar las validaciones de negocio críticas, como comprobar que la asignación de una sala a una actividad no solape sus horarios con otras reservas existentes antes de persistir los datos en la base de datos.
- **Gestión Usuarios** [Component: Python / Django]: Componente encargado de la autenticación, autorización y gestión de sesiones. Controla el acceso basado en roles (RBAC), asegurando que el *Personal Administrativo*, los *Monitores* y los *Usuarios Inscritos* solo puedan ejecutar acciones y consultar datos permitidos para su nivel de privilegios.
- **Gestión Inscripciones** [Component: Python / Django]: Procesa las transacciones transaccionales del sistema. Recibe las peticiones de reserva, verifica atómicamente la disponibilidad de aforo en la actividad solicitada y registra la inscripción. Su diseño está altamente cohesionado con el modelo de datos para garantizar la integridad de las plazas.

- Manejador de Signals [Component: Python / Django (Observer)]: Implementación práctica del patrón de diseño *Observer*. Utiliza el framework de señales de Django (ej. @receiver) para escuchar eventos emitidos por el módulo de *Gestión Inscripciones* (como una nueva alta o baja). Al reaccionar, delega las tareas secundarias al sistema de notificaciones, garantizando un acoplamiento nulo entre la transacción de base de datos y los servicios externos.

Componentes del Contenedor: Front-end (Capa de Presentación):

Estos componentes representan la capa *Template* del patrón MVT, combinados con la lógica estática del cliente. Renderizan la información preparada por el Back-end:

- Vista Actividades [Component: HTML / CSS / JS]: Interfaz gráfica dedicada a la visualización del catálogo. Presenta a los usuarios el listado de talleres disponibles, filtrado por fechas o categorías, consumiendo la información procesada por la *Gestión de Actividades*.
- Vista Inscripciones [Component: HTML / CSS / JS]: Formularios interactivos y paneles de control del usuario. Permite a los ciudadanos solicitar plazas y visualizar su historial de reservas de forma intuitiva, enviando los datos estructurales (*payloads*) al componente de *Gestión Inscripciones* del backend.
- Administración Monitores [Component: HTML / CSS / JS]: Interfaz restringida para los profesionales del centro. Genera las vistas en las que los monitores consultan sus horarios asignados, las salas donde deben impartir clase y las listas de asistentes de sus respectivos grupos.
- Login / Signin [Component: HTML / CSS / JS]: Punto de entrada y seguridad visual del sistema. Captura las credenciales de los usuarios e interactúa directamente con la *Gestión de Usuarios* para establecer los *tokens* o *cookies* de sesión requeridos para navegar por la plataforma.

Contenedores Externos Interactuantes:

(Se mencionan en este nivel para justificar las dependencias y salidas de datos de los componentes internos)

- BD [Container: Postgres]: Sistema gestor de base de datos relacional. Recibe las sentencias SQL generadas por el ORM de Django desde los componentes de gestión para garantizar la persistencia a largo plazo y la integridad de las relaciones (actividades-salas-usuarios).
- Controlador Notificaciones [Container: Python]: Proceso *worker* asíncrono. Actúa como sumidero de los eventos disparados por el *Manejador de Signals*, asumiendo la responsabilidad exclusiva de comunicarse con la API externa de AWS SNS sin penalizar el tiempo de respuesta del hilo principal del Back-end.

2. Justificación de decisiones arquitectónicas

El sistema se ha diseñado siguiendo el patrón MVT (Model-View-Template) propio del framework Django, que ofrece una separación clara de responsabilidades entre la lógica de negocio, el acceso a datos y la presentación.

La elección de Django como framework principal se justifica por varias razones. En primer lugar, incorpora de serie un ORM potente que permite definir las relaciones entre entidades (1-a-1, 1-a-N y N-a-N) directamente en Python, sin necesidad de escribir SQL. En segundo lugar, el sistema de URLs y vistas permite estructurar todos los endpoints requeridos de forma limpia y mantenible. En tercer lugar, su sistema de formularios con `ModelForm` simplifica la validación y el procesamiento de datos de entrada, reduciendo el código repetitivo. Por último, Django cuenta con una comunidad amplia, documentación exhaustiva y un ciclo de desarrollo rápido, aspectos especialmente relevantes en un proyecto académico con tiempo limitado.

Diseño modular: el proyecto se organiza en una única aplicación Django (`gestion`) dado que todas las entidades pertenecen al mismo dominio funcional y tienen un alto grado de cohesión entre ellas. Esta decisión simplifica la estructura sin sacrificar claridad.

Además, siguiendo las mejores prácticas del framework, se ha adoptado la filosofía "*Fat Models, Thin Views*" (Modelos robustos, Vistas delgadas). Esto implica que la lógica de validación de negocio (como verificar la disponibilidad de aforo o si una sala está libre en un horario concreto) reside en el Modelo o en los Formularios, manteniendo las Vistas limpias y dedicadas exclusivamente a coordinar el flujo de la petición HTTP. En caso de que el sistema creciera con funcionalidades independientes (como un módulo de pagos o notificaciones complejas), se separarían en aplicaciones adicionales.

Base de datos: se utiliza SQLite durante el desarrollo por no requerir instalación de servidor y ser suficiente para pruebas locales. Para un entorno de producción real se migraría a PostgreSQL, que ofrece mayor robustez, soporte nativo de tipos avanzados y mejor gestión de la concurrencia. Asimismo, PostgreSQL garantiza una estricta integridad referencial, lo cual es crítico en este sistema para evitar fallos lógicos graves (como la eliminación en cascada accidental de registros históricos o la asignación de actividades a salas inexistentes) mediante el uso de restricciones (*constraints*) a nivel de base de datos.

Aunque el core del sistema es un monolito en Django, el envío de notificaciones se ha extraído a un contenedor independiente. Esto garantiza que si la API de AWS SNS tarda en responder, la aplicación web no se bloquea y el usuario recibe confirmación inmediata en la pantalla. Es una arquitectura híbrida orientada a mejorar el rendimiento y la experiencia de usuario.

3. Patrón de diseño: Observer

El patrón elegido es el Observer (Publicador-Suscriptor), perteneciente a la categoría de patrones de comportamiento del catálogo de *Gang of Four*.

Problema que resuelve: cuando un usuario se inscribe en una actividad, el sistema debe reaccionar de forma coordinada: actualizar el número de plazas disponibles, notificar al monitor asignado y registrar el evento en un log de auditoría. Si toda esta lógica se implementa directamente en la vista de inscripción, el código queda altamente acoplado y cualquier nueva reacción obliga a modificar esa vista, violando el principio *Open/Closed*.

Aplicación del patrón: la vista de inscripción actúa como *publisher* y emite un evento cuando se produce una nueva inscripción. Los distintos *subscribers* (actualizador de plazas, notificador, logger) se suscriben a ese evento de forma independiente y reaccionan sin que el *publisher* conozca su existencia. Esto proporciona una gran asincronía futura: el día de mañana se puede añadir un nuevo suscriptor (por ejemplo, para enviar un SMS o imprimir un carnet) sin tocar una sola línea del código original de la inscripción.

Relación con Django: el framework implementa este patrón de forma nativa mediante su sistema de *signals*. La señal `m2m_changed` se dispara automáticamente cuando cambia una relación ManyToMany (como la inscripción de un usuario a una actividad), y cualquier función decorada con `@receiver` actúa como *subscriber*. Esto hace que el Observer no sea solo un patrón teórico aplicable, sino la forma idiomática de resolver este problema concreto en Django.

3. MVC vs MVT

MVC (Model-View-Controller) es un patrón arquitectónico que divide una aplicación en tres componentes con responsabilidades bien diferenciadas. El Model encapsula los datos y la lógica de negocio, siendo el único componente que interactúa con la base de datos. La View es responsable de la presentación, es decir, de generar la interfaz que el usuario ve. El Controller actúa como intermediario: recibe las peticiones del usuario, las procesa coordinando Model y View, y devuelve una respuesta.

MVT (Model-View-Template) es la variante que implementa Django. La nomenclatura es diferente pero el concepto es equivalente. El Model (`models.py`) cumple exactamente la misma función que en MVC: define las entidades y gestiona el acceso a datos mediante el ORM. La View de Django (`views.py`) equivale en realidad al Controller de MVC: recibe la petición HTTP, consulta los modelos necesarios, aplica la lógica de negocio y selecciona qué template renderizar. El Template (archivos `.html`) equivale a la View de MVC: se ocupa exclusivamente de presentar los datos al usuario.

La diferencia más relevante es que Django añade una capa adicional explícita que MVC no contempla como componente separado: el URL dispatcher (`urls.py`), que actúa como un frontal que analiza la URL de cada petición entrante y decide qué View debe procesarla. En frameworks MVC tradicionales esta responsabilidad suele estar integrada en el propio Controller o en un enrutador implícito del framework.

En la práctica, para este proyecto la correspondencia es la siguiente: los archivos `models.py` definen las entidades (Actividad, Usuario, Monitor, Sala, ResponsableSala); los archivos `views.py` contienen las funciones o clases que procesan cada petición y preparan el contexto de datos; los archivos `.html` en la carpeta `templates/` renderizan la información para el usuario; y el archivo `urls.py` conecta cada *endpoint* con su vista correspondiente.